

Numba



KHOA CÔNG NGHỆ THÔNG TIN
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

fit@hcmus

Numba overview

- Numba is a also Python lib which helps speed up Python program when working with large amount of numerical data
- But Numba solves Numpy's disadvantages
 - ▣ Numba allows to use loops with Numpy arrays
 - ▣ Numba allows to run parallel on CPU as well as GPU

Numba is

Function compiler:

- ❑ Numba compiles Python functions, not entire applications, and not parts of functions.
- ❑ Numba does not replace your Python interpreter, but is just another Python module that can turn a function into a (usually) faster function.

Type-specializing:

- ❑ Numba speeds up your function by generating a specialized implementation for the specific data types you are using.
- ❑ Python functions are designed to operate on generic data types, which makes them very flexible, but also very slow.
- ❑ In practice, you only will call a function with a small number of argument types, so Numba will generate a fast implementation for each set of types.

Numba is

Just-in-time:

- ❑ Numba translates functions when they are first called.
- ❑ This ensures the compiler knows what argument types you will be using.
- ❑ This also allows Numba to be used interactively in a Jupyter notebook just as easily as a traditional application

Numerically-focused:

- ❑ Currently, Numba is focused on numerical data types, like int, float, and complex.
- ❑ There is very limited string processing support, and many string use cases are not going to work well on the GPU.
- ❑ To get best results with Numba, you will likely be using NumPy arrays.

Numba overview

- ❑ Numba helps speed up a Python program by
- ❑ compiling Python functions into machine code by
- ❑ adding decorators before Python functions
- ❑ Decorator is a function which takes some function *f* as input, “decorate” (modify the content) *f* and return decorated *f*

@make_faster

def f:

...

is equivalent with

f = make_faster(f)

Numba overview

- Numba helps speed up a Python program by
- compiling Python functions into machine code by
- adding decorators before Python functions
- Decorator is a function which takes some function *f* as input, “decorate” (modify the content) *f* and return decorated *f*

@make_faster

def f:

...

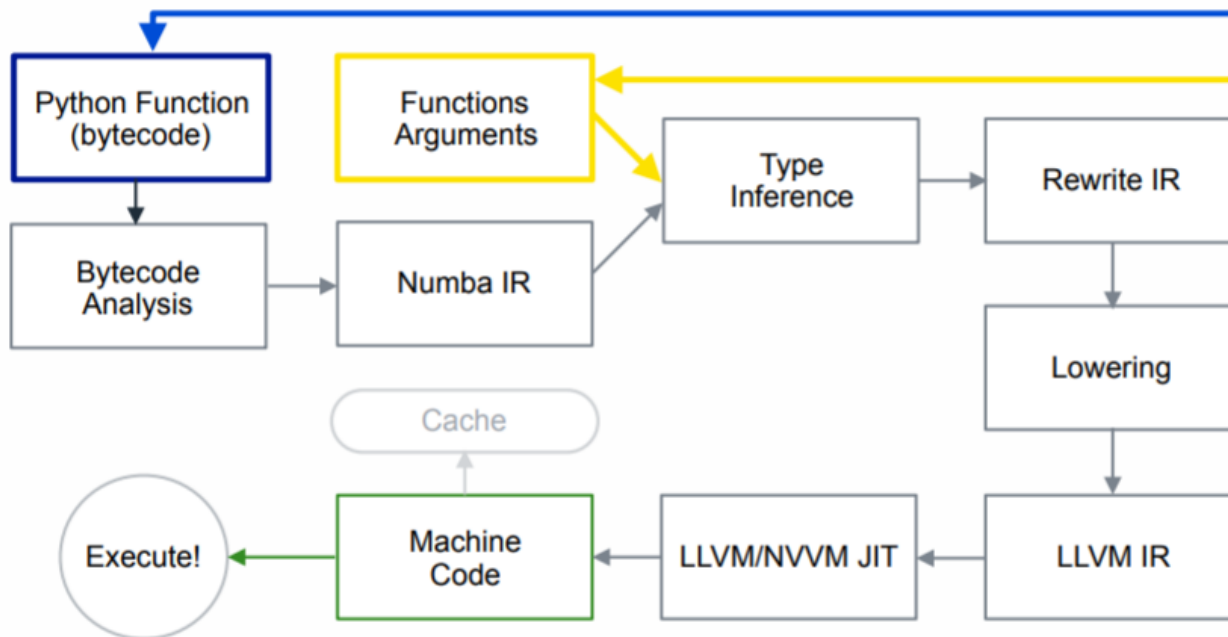
is equivalent with

f = make_faster(f)

Numba overview

How Does Numba Work?

```
@jit
def do_math(a, b):
    ...
>>> do_math(x, y)
```



Numba overview

- For Numba to be able to help speed up, a Python function needs to **satisfy some conditions**
- For example:
 - ▣ Numba will help speed up a lot with a Python function having Numpy arrays + loops
 - ▣ Numba cannot compile a Python function having a list with both numbers and strings
- For more information, you can read Numba document: [“Supported Python features”](#) and [“Supported NumPy features”](#)

Sequential version

- ❑ Numba provides different decorators to compile a Python function in different ways
- ❑ **@jit**: Compile a Python function into machine code and run sequentially on CPU
 - ❑ Often require little code modification
- ❑ **@jit(parallel=True)**: Compile a Python function into machine code and run parallel on CPU
 - ❑ Often require more code modification than the way above
 - ❑ Currently, just allow to parallelize in a simple way (independent threads, no cooperation)
- ❑ **@cuda.jit**: Compile a Python function into machine code and run parallel on GPU
 - ❑ Often require a lot of code modification
 - ❑ Allow to parallelize in a complex way (threads can cooperate with each other) ...

Reference

- ☐ <https://numba.pydata.org/numba-doc/latest/cuda/overview.html>
- ☐ <https://github.com/ContinuumIO/gtc2017-numba>